大家好，我是 Guide！在构建 AI 应用的过程中，选择一个合适的 AI 框架至关重要。今天我要带大家深入了解 **Spring AI**——Spring 官方打造的 AI 应用开发框架，并通过本项目的实际实现，展示如何在 Spring Boot 中优雅地集成大模型能力。

Spring AI 简介

Spring AI 是 Spring 官方打造的 AI 应用开发框架，旨在将 AI 能力无缝集成到 Spring 生态系统中。它侧重于提供构建 AI 应用所需的**底层原子能力抽象**，让 Java 开发者可以像使用其他 Spring 组件一样，轻松地与大语言模型进行交互。

能力	说明	本项目应用场景
模型通信 (ChatClient)	统一接口与不同 LLM（OpenAI、Ollama、通义千问等）对话	简历评分、面试问题生成、面试评估、知识库问答
提示词 (Prompt)	结构化管理发送给模型的提示词	使用 `.st` 模板文件管理提示词
RAG (检索增强生成)	通过 `VectorStore` 实现 RAG 模式	知识库问答，详见 [知识库 RAG 问答]()
工具调用 (Function Calling)	模型调用 Java 应用中定义的方法	-
记忆 (ChatMemory)	管理多轮对话的上下文历史	RAG 聊天会话管理

> **注意**：Spring AI 本身未提供多智能体（Multi-agent）开发能力。若需在 Spring AI 项目中开发多智能体应用，可以考虑集成 **LangGraph4j**（Java 版 LangGraph）。

为何选择 Spring AI

如果你的项目基于 Spring Boot 技术栈，并且希望获得官方的长期支持、紧跟 Spring 生态发展步伐，那么可以优先考虑 Spring AI。

核心优势

优势	说明
无缝集成	与现有 Spring Boot 技术栈完美融合，学习成本低，能快速上手
高度抽象	提供统一的、与具体模型无关的 API，轻松在 OpenAI、Ollama、通义千问之间切换
生态整合	整合向量数据库（Chroma, PGVector）、ETL 框架等 AI 应用开发所需的整个工具链

Spring 生态中的 AI 框架对比

目前，Java 依然是企业级开发的主流，市面上有多款 AI 应用开发框架：

框架	特点	JDK 基线	框架绑定	适用场景
Spring AI	Spring 官方，原生集成	**JDK 17+**（建议 JDK 21）	**Spring Boot 3.2+**	Spring 项目首选，未来主流
LangChain4j	功能全面，更新快	JDK 8+	无限制（Boot 2.x/3.x）	老项目，兼容性需求
Solon-AI	轻量化，性能优秀	JDK 8+	Solon 生态（也支持 Boot）	Serverless、GraalVM 原生镜像
Agent-Flex	专注于 Agent 编排	JDK 11/17+	无框架依赖	Agent 编排需求为主

Spring AI 的杀手锏：它是 Spring 生态的"原生延伸"。将 AI 能力抽象成了标准组件，就像我们熟悉的 `JdbcTemplate` 或 `RestTemplate` 一样。对于老 Spring 玩家来说，接入成本几乎为零。

聚焦 90% 的核心落地场景

技术要为业务服务，不要为了炫技而引入复杂性。绝大多数商业场景不需要你突破算法，而是要解决这四个痛点：**大模型调用、RAG（检索增强生成）、结构化输出以及流式响应。**

Spring AI 2.0 在这些领域已经扎得足够深：

- **RAG 自动化**：对向量数据库（Redis, Milvus, pgvector）的抽象极其优雅，真正实现了"代码解耦"。
- **性能榨取**：全面拥抱 **Java 21 虚拟线程（Virtual Threads）**，在处理高并发 AI 请求时，吞吐量提升明显。
- **工程规范**：配合 Spring Boot 4.0 和 Gradle Version Catalog，让 AI 模块的维护成本显著降低。

项目依赖配置

本项目采用 **Spring Boot 4.0 + Java 21 + Spring AI 2.0** 技术栈，使用阿里云 DashScope（通义千问）作为大模型服务提供商。

添加依赖

```
// build.gradle
dependencies {
    // Spring AI 2.0 - OpenAI兼容模式 (阿里云DashScope)
    implementation "org.springframework.ai:spring-ai-starter-model-openai:${libs.versions.spring.ai.get()}"
    // Spring AI 2.0 - PostgreSQL Vector Store (pgvector)
    implementation "org.springframework.ai:spring-ai-starter-vector-store-pgvector:${libs.versions.spring.ai.get()}"
}
```

版本说明：

- `spring-ai-starter-model-openai`：提供 `ChatClient` 和 `EmbeddingModel`
- `spring-ai-starter-vector-store-pgvector`：提供 `VectorStore` 对接 PostgreSQL + pgvector

配置属性

```yaml
# application.yml
spring:
  ai:
    openai:
      base-url: https://dashscope.aliyuncs.com/compatible-mode
      api-key: ${AI_BAILIAN_API_KEY}
      chat:
        options:
          model: ${AI_MODEL:qwen-plus}
          temperature: 0.2
      # Embedding模型配置
      embedding:
        options:
          model: text-embedding-v3
    # 禁用自动重试机制，让异常立即返回
    retry:
      max-attempts: 1
      on-client-errors: false
    # PostgreSQL Vector Store配置
    vectorstore:
      pgvector:
        index-type: HNSW
        distance-type: COSINE_DISTANCE
        dimensions: 1024  # text-embedding-v3实际生成的向量维度
        initialize-schema: true
        remove-existing-vector-store-table: false
app:
  ai:
    structured-max-attempts: ${APP_AI_STRUCTURED_MAX_ATTEMPTS:2}
    structured-include-last-error: ${APP_AI_STRUCTURED_INCLUDE_LAST_ERROR:true}
```

关键配置说明:

配置项	说明
`base-url`	阿里云 DashScope 的 OpenAI 兼容端点
`api-key`	通过环境变量注入，生产环境**严禁硬编码**
`temperature`	控制输出随机性（0-1），本项目使用 0.2（更利于结构化输出稳定）
`dimensions`	text-embedding-v3 的向量维度为 1024
`initialize-schema`	开发环境设为 true 自动创建表，生产环境设为 false
`app.ai.structured-max-attempts`	业务层结构化输出重试次数（默认 2）
`app.ai.structured-include-last-error`	重试时是否注入上次失败原因（默认 true）

大模型核心概念（建议先读）

为了避免本文变成“概念大全”，关于 **Token / 上下文窗口 / Temperature & Top-p / 结构化输出失败路径 / RAG 召回评估 / 429 限流与重试** 等内容，我把它们独立整理成了另一篇文章，并且用本项目的配置与代码做了锚点说明：[大模型核心概念详解（面向 Spring AI 集成）](#)。

本文后续将专注 Spring AI 在本项目中的**落地实现**：`ChatClient` 调用方式、`.st` 提示词模板组织、`BeanOutputConverter` 结构化输出等。

ChatClient：统一的大模型调用接口

`ChatClient` 是 Spring AI 提供的流式 API，用于与大语言模型进行对话。本项目在四个核心场景中使用 ChatClient：

1. **简历评分**：`ResumeGradingService` - 分析简历内容并给出评分和建议
2. **面试问题生成**：`InterviewQuestionService` - 基于简历生成个性化面试问题（含追问）
3. **面试评估**：`AnswerEvaluationService` - 评估面试回答并生成报告（含分批评估与二次总结）
4. **RAG 问答**：`KnowledgeBaseQueryService` - 基于知识库回答用户问题

服务类设计

以简历评分服务为例：

```
@Service
public class ResumeGradingService {

    private static final Logger log =
LoggerFactory.getLogger(ResumeGradingService.class);

    private final ChatClient chatClient;
    private final PromptTemplate systemPromptTemplate;
    private final PromptTemplate userPromptTemplate;
    private final BeanOutputConverter<ResumeAnalysisResponseDTO>
outputConverter;
    private final StructuredOutputInvoker structuredOutputInvoker;

    // 中间DTO用于接收AI响应（字段名与提示词中要求的 JSON 结构一致，BeanOutputConverter
按字段名解析）
    private record ResumeAnalysisResponseDTO(
        int overallScore,
        ScoreDetailDTO scoreDetail,
        String summary,
        List<String> strengths,
        List<SuggestionDTO> suggestions
    ) {}

    private record ScoreDetailDTO(
        int contentScore,
        int structureScore,
        int skillMatchScore,
        int expressionScore,
        int projectScore
    ) {}

    private record SuggestionDTO(String category, String priority, String issue,
String recommendation) {}

    public ResumeGradingService(
            ChatClient.Builder chatClientBuilder,
            StructuredOutputInvoker structuredOutputInvoker,
            @Value("classpath:prompts/resume-analysis-system.st") Resource
systemPromptResource,
```

```
            @Value("classpath:prompts/resume-analysis-user.st") Resource
userPromptResource) throws IOException {
        this.chatClient = chatClientBuilder.build();
        this.structuredOutputInvoker = structuredOutputInvoker;
        this.systemPromptTemplate = new
PromptTemplate(systemPromptResource.getContentAsString(StandardCharsets.UTF_8));
        this.userPromptTemplate = new
PromptTemplate(userPromptResource.getContentAsString(StandardCharsets.UTF_8));
        this.outputConverter = new BeanOutputConverter<>
(ResumeAnalysisResponseDTO.class);
    }
}
```

设计要点:

1. **构造器注入**: 通过 `ChatClient.Builder` 构建 ChatClient, `StructuredOutputInvoker` 封装结构化输出的重试策略
2. **提示词模板化**: 使用 `@Value` 注入 `.st` 提示词文件
3. **结构化输出**: 使用 `BeanOutputConverter` 将 AI 响应转换为 Java 对象

调用大模型

```
public ResumeAnalysisResponse analyzeResume(String resumeText) {
    log.info("开始分析简历，文本长度: {} 字符", resumeText.length());

    try {
        // 1. 加载系统提示词
        String systemPrompt = systemPromptTemplate.render();

        // 2. 加载用户提示词并填充变量
        Map<String, Object> variables = new HashMap<>();
        variables.put("resumeText", resumeText);
        String userPrompt = userPromptTemplate.render(variables);

        // 3. 添加格式指令到系统提示词
        String systemPromptWithFormat = systemPrompt + "\n\n" +
outputConverter.getFormat();

        // 4. 调用AI（StructuredOutputInvoker 支持解析失败时按配置重试并注入上次错误信
息）
        ResumeAnalysisResponseDTO dto;
        try {
            dto = structuredOutputInvoker.invoke(
                chatClient,
                systemPromptWithFormat,
                userPrompt,
                outputConverter,
                ErrorCode.RESUME_ANALYSIS_FAILED,
                "简历分析失败: ",
                "简历分析",
                log
            );
            log.debug("AI响应解析成功: overallScore={}", dto.overallScore());
        } catch (Exception e) {
            log.error("简历分析AI调用失败: {}", e.getMessage(), e);
            throw new BusinessException(ErrorCode.RESUME_ANALYSIS_FAILED, "简历分
析失败: " + e.getMessage());
```

```
        }

        // 5. 转换为业务对象
        ResumeAnalysisResponse result = convertToResponse(dto, resumeText);
        log.info("简历分析完成，总分: {}", result.overallScore());

        return result;

    } catch (Exception e) {
        log.error("简历分析失败: {}", e.getMessage(), e);
        return createErrorResponse(resumeText, e.getMessage());
    }
}
```

关键点：格式指令由业务代码通过 `systemPrompt + "\n\n" + outputConverter.getFormat()` 拼接到系统提示词后传入；AI 响应由 `ChatClient.call().entity(outputConverter)` 经 `BeanOutputConverter` 解析为指定的 Java Record 类型。

调用流程图：

```
┌──────────────┐      ┌──────────────┐      ┌──────────────┐
│ 加载提示词   │──────▶│ 填充变量     │──────▶│ 调用 ChatClient│
└──────────────┘      └──────────────┘      └──────────────┘
                                                    │
                                                    ▼
┌──────────────┐      ┌──────────────┐      ┌──────────────┐
│ 转换业务对象 │◀──────│ 解析响应     │◀──────│ 获取 AI 响应 │
└──────────────┘      └──────────────┘      └──────────────┘
```

面试问题生成：智能追问系统

面试问题生成服务基于候选人简历生成针对性的面试问题，并支持可配置的追问数量。

服务类设计

```
@Service
public class InterviewQuestionService {

    private static final Logger log =
LoggerFactory.getLogger(InterviewQuestionService.class);

    private final ChatClient chatClient;
    private final PromptTemplate systemPromptTemplate;
    private final PromptTemplate userPromptTemplate;
    private final BeanOutputConverter<QuestionListDTO> outputConverter;
    private final StructuredOutputInvoker structuredOutputInvoker;
    private final int followUpCount;  // 可配置的追问数量

    private record QuestionListDTO(List<QuestionDTO> questions) {}

    private record QuestionDTO(
        String question,
        String type,
        String category,
        List<String> followUps  // 每个主问题的追问列表
```

```
    ) {}

    public InterviewQuestionService(
            ChatClient.Builder chatClientBuilder,
            StructuredOutputInvoker structuredOutputInvoker,
            @Value("classpath:prompts/interview-question-system.st") Resource
systemPromptResource,
            @Value("classpath:prompts/interview-question-user.st") Resource
userPromptResource,
            @Value("${app.interview.follow-up-count:1}") int followUpCount)
throws IOException {
        this.chatClient = chatClientBuilder.build();
        this.structuredOutputInvoker = structuredOutputInvoker;
        this.systemPromptTemplate = new
PromptTemplate(systemPromptResource.getContentAsString(StandardCharsets.UTF_8));
        this.userPromptTemplate = new
PromptTemplate(userPromptResource.getContentAsString(StandardCharsets.UTF_8));
        this.outputConverter = new BeanOutputConverter<>(QuestionListDTO.class);
        this.followUpCount = Math.max(0, Math.min(followUpCount,
MAX_FOLLOW_UP_COUNT));
    }
}
```

> 模拟面试的完整流程（会话状态、历史题去重、Redis 缓存与持久化等）见模拟面试功能实现，此处仅说明 Spring AI 侧的出题实现。

追问生成配置

通过配置文件控制每个主问题生成的追问数量：

```
app:
  interview:
    follow-up-count: ${APP_INTERVIEW_FOLLOW_UP_COUNT:1}  # 默认 1 条，最多 2 条
```

追问展开为线性问答流

生成的追问会被展开为线性问答流，每个追问都会成为独立的问题记录：

```
private List<InterviewQuestionDTO> convertToQuestions(QuestionListDTO dto) {
    List<InterviewQuestionDTO> questions = new ArrayList<>();
    int index = 0;

    if (dto == null || dto.questions() == null) {
        return questions;
    }

    for (QuestionDTO q : dto.questions()) {
        if (q == null || q.question() == null || q.question().isBlank()) {
            continue;
        }
        QuestionType type = parseQuestionType(q.type());  // AI 返回的 type 为字符
串，需解析为枚举
        int mainQuestionIndex = index;
        questions.add(InterviewQuestionDTO.create(
            index++,
            q.question(),
```

```
            type,
            q.category(),
            false,
            null
        ));

        // 展开追问为独立问题
        List<String> followUps = sanitizeFollowUps(q.followUps());
        for (int i = 0; i < followUps.size(); i++) {
            questions.add(InterviewQuestionDTO.create(
                index++,
                followUps.get(i),
                type,
                buildFollowUpCategory(q.category(), i + 1),
                true,
                mainQuestionIndex
            ));
        }
    }

    return questions;
}
```

当前实现中，追问关系使用结构化字段标记：

- `isFollowUp`：是否追问；
- `parentQuestionIndex`：追问关联的主问题索引。

这两个字段会写入会话的 `questionsJson`，用于后续历史题去重与评估聚合。

历史问题注入与去重

为避免生成重复问题，系统在创建新会话时会由持久化/会话层从 `InterviewSessionRepository.findTop10ByResumeIdOrderByCreatedAtDesc(resumeId)` 读取最近 10 个会话的题目 JSON，解析出主问题（排除追问）后去重并限制条数，注入到出题 Prompt 的 `historicalQuestions` 变量。

问题生成流程

```
┌──────────────────┐      ┌──────────────────┐      ┌──────────────────┐
│ 分析简历内容      │──────▶│ 计算问题分布      │──────▶│ 生成主问题+追问   │
└──────────────────┘      └──────────────────┘      └──────────────────┘
                                                            │
                                                            ▼
┌──────────────────┐      ┌──────────────────┐      ┌──────────────────┐
│ 返回线性问题流    │◀──────│ 追问展开          │◀──────│ AI 生成问题       │
└──────────────────┘      └──────────────────┘      └──────────────────┘
```

提示词模板示例（`interview-question-user.st`）：

```
# Input Data
请根据以下候选人简历内容，生成共 {questionCount} 个主问题。
每个主问题必须额外生成 {followUpCount} 个追问。

## 问题分布要求
| 类型 | 数量 | 说明 |
```

```
|------|------|------|
| 项目经历 (PROJECT) | {projectCount} 题 | 基于简历中的具体项目深度提问 |
| MySQL | {mysqlCount} 题 | 索引、事务、SQL 优化等 |
| Redis | {redisCount} 题 | 数据结构、缓存策略、分布式锁等 |
| Java 基础 (JAVA_BASIC) | {javaBasicCount} 题 | 面向对象、JVM、异常处理等 |
| Java 集合 (JAVA_COLLECTION) | {javaCollectionCount} 题 | 集合框架、源码原理等 |
| Java 并发 (JAVA_CONCURRENT) | {javaConcurrentCount} 题 | 线程、锁、并发工具类等 |
| Spring/Spring Boot | {springCount} 题 | IoC/AOP、自动配置等 |
```

面试评估：分批评估与二次总结

面试评估服务采用**分批评估 + 二次总结**的两阶段架构，有效规避大模型 Token 溢出风险，同时保证评估结果的一致性。**模拟面试的完整业务架构（会话状态、缓存与持久化、Redis Stream 异步评估流水线等）见**模拟面试功能实现**；本文仅说明 Spring AI 侧的 Prompt 与结构化输出实现。**

服务类设计

```java
@Service
public class AnswerEvaluationService {

    private static final Logger log =
LoggerFactory.getLogger(AnswerEvaluationService.class);

    private final ChatClient chatClient;
    private final PromptTemplate systemPromptTemplate;
    private final PromptTemplate userPromptTemplate;
    private final BeanOutputConverter<EvaluationReportDTO> outputConverter;
    private final PromptTemplate summarySystemPromptTemplate;
    private final PromptTemplate summaryUserPromptTemplate;
    private final BeanOutputConverter<FinalSummaryDTO> summaryOutputConverter;
    private final StructuredOutputInvoker structuredOutputInvoker;
    private final int evaluationBatchSize;  // 分批评估大小

    public AnswerEvaluationService(
            ChatClient.Builder chatClientBuilder,
            StructuredOutputInvoker structuredOutputInvoker,
            @Value("classpath:prompts/interview-evaluation-system.st") Resource
systemPromptResource,
            @Value("classpath:prompts/interview-evaluation-user.st") Resource
userPromptResource,
            @Value("classpath:prompts/interview-evaluation-summary-system.st")
Resource summarySystemPromptResource,
            @Value("classpath:prompts/interview-evaluation-summary-user.st")
Resource summaryUserPromptResource,
            @Value("${app.interview.evaluation.batch-size:8}") int
evaluationBatchSize) throws IOException {
        this.chatClient = chatClientBuilder.build();
        this.structuredOutputInvoker = structuredOutputInvoker;
        // ... 初始化各 PromptTemplate 与 OutputConverter
        this.evaluationBatchSize = Math.max(1, evaluationBatchSize);
    }
}
```

分批评估配置

```yaml
app:
  interview:
    evaluation:
      batch-size: ${APP_INTERVIEW_EVALUATION_BATCH_SIZE:8}  # 默认每批 8 题
```

分批评估流程

```
┌──────────────────────────────────────────────────────────────────────┐
│                          第一阶段：分批评估                          │
├──────────────────────────────────────────────────────────────────────┤
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐             │
│   │  批次 1      │   │  批次 2      │   │  批次 N      │             │
│   │  (1-8题)     │   │  (9-16题)    │   │  (...)       │             │
│   └──────┬───────┘   └──────┬───────┘   └──────┬───────┘             │
│          │                  │                  │                     │
│          └──────────────────┴──────────────────┘                     │
│                             ▼                                        │
│                 合并分批结果（降级方案）                              │
└──────────────────────────────────┬───────────────────────────────────┘
                                   │
                                   ▼
┌──────────────────────────────────────────────────────────────────────┐
│                       第二阶段：二次总结（可选）                     │
├──────────────────────────────────────────────────────────────────────┤
│  输入：类别得分概览 + 题目评估高亮 + 分批初始结论                    │
│                             ▼                                        │
│                 AI 生成最终汇总评估                                  │
│                             ▼                                        │
│              成功：使用二次总结结果  |  失败：降级到分批聚合          │
└──────────────────────────────────────────────────────────────────────┘
```

优雅降级机制

二次总结阶段通过 `StructuredOutputInvoker` 调用 ChatClient；若解析失败则按配置重试，仍失败则降级为分批聚合结果（使用各批次已有的 `overallFeedback`、`strengths`、`improvements` 合并），保证评估结果可用。详见 [模拟面试功能实现]。

降级策略优势：

- **容错性强**：即使二次总结失败，仍有可用的评估结果
- **用户体验好**：避免因 AI 调用异常导致的评估完全失败
- **结果可靠**：分批聚合结果是各批次评估的直接合并，保证了基本准确性

Prompt 管理：提示词工程

本项目使用 `.st`（String Template）文件管理提示词，放置在 `app/src/main/resources/prompts/` 目录下。

提示词文件结构

```
prompts/
├── resume-analysis-system.st              # 简历分析系统提示词
├── resume-analysis-user.st                # 简历分析用户提示词
├── interview-question-system.st           # 面试问题生成系统提示词
├── interview-question-user.st             # 面试问题生成用户提示词
├── interview-evaluation-system.st         # 面试评估系统提示词
├── interview-evaluation-user.st           # 面试评估用户提示词
├── interview-evaluation-summary-system.st # 面试评估二次总结系统提示词
├── interview-evaluation-summary-user.st   # 面试评估二次总结用户提示词
├── knowledgebase-query-system.st          # 知识库问答系统提示词
├── knowledgebase-query-user.st            # 知识库问答用户提示词
└── knowledgebase-query-rewrite.st         # 知识库查询改写提示词（短 query 扩写）
```

提示词模板示例

系统提示词（`resume-analysis-system.st`）：

```
# Role
你是一位拥有 10 年以上经验的资深技术架构师、工程管理专家及高级技术人才顾问。你具备跨语言（Java, Go, Python, Rust, Frontend, Infrastructure 等）的深度技术视野，擅长从底层架构、工程效率和业务价值三个维度对简历进行“穿透式”审计。
# Task
请对用户提供的简历内容进行深度技术审计、多维度评分，并提供极具实操性的改进建议，特别是针对“项目经历”的重写与优化。

# Project Audit Standards (项目审计标准)
在审计项目经历时，必须参考以下准则：
1. **技术选型合理性**：识别并纠正不合理的方案（例如：本地缓存应优先推荐 Caffeine 而非 HashMap；分布式锁应推荐 Redisson；复杂异步编排应使用 CompletableFuture）。
2. **业务场景融合**：拒绝纯技术堆砌。描述必须遵循“技术实现 + 业务场景 + 结果量化”的模式。
3. **表达精炼度**：单条描述建议不超过两行。动词开头（主导、优化、解决、搭建），删除“负责...的开发”等冗余词汇。
4. **深度技术点**：优先挖掘 JVM 调优、多线程并发、分布式一致性、性能瓶颈解决等高价值信息。

# Scoring Rubrics (Total: 100)
1. **projectTechDepth (0-40分)**：是否避开了烂大街的项目（如博客、外卖）。是否体现了复杂问题排查（死锁、调优）或成熟中间件的深度运用。技术是否解决了实际业务痛点。是否有清晰的业务闭环描述。是否有明确的量化产出（如：响应时间从 2s 降至 0.2s，QPS 提升 5 倍）。
2. **skillMatchScore (0-20分)**：技术栈专业度。区分“了解/熟悉/熟练掌握”（尽量不要用精通），核心能力（高并发、分布式）是否突出，是否满足对应岗位要求。
3. **contentScore (0-15分)**：模块顺序是否合理？简历信息展示建议的顺序为（个人建议，可根据自身情况动态调整）：个人信息-> 求职意向（可包含在个人信息中）-> 教育经历 -> 专业技能 ->工作/实习经历 -> 项目经历 ->证书奖项（可选）->校园经历 (可选) ->个人评价/工作期望（真诚即可，别说太多虚的）。
4. **structureScore (0-15分)**：技术名词大小写必须绝对规范（如 Java, Spring Boot, MySQL, Redis, GitHub）。
5. **expressionScore (0-10分)**：语言是否简洁，是否有过多不专业的词汇表达。

# Audit Workflow
1. **名词纠错**：扫描全文，列出所有不规范的技术名词。
2. **深度重写 (Deep Rewrite)**：从简历中挑选 2-3 条核心项目描述，基于 STAR 法则和提供的【优秀模板】进行对比重写。
3. **方案优化建议**：针对用户简历中平庸的技术方案，给出更具竞争力的替代方案建议。

# Constraints
- 必须输出严谨的 JSON 格式。
```

```
- 严禁虚构简历中不存在的业务背景，但可以基于现有背景建议合理的量化指标。
- 建议必须具有可操作性，提供"原句 vs 优化句"的对比。

# Output Format
请直接输出一个 JSON 对象，不要包含 Markdown 代码块标签（如 ```json ）。

JSON 结构必须严格包含以下字段：
1. overallScore: 整数，总分（0-100）。
2. scoreDetail: 一个对象，包含以下五个整数字段：
   - projectScore: 项目经验评分（0-40分）
   - skillMatchScore: 技能匹配度评分（0-20）
   - contentScore: 内容完整性评分（0-15）
   - structureScore: 结构清晰度评分（0-15）
   - expressionScore: 表达专业性评分（0-10）
3. summary: 字符串，一句话总结简历的整体情况。
4. strengths: 字符串数组，列出简历的优势点。
5. suggestions: 对象数组，每个对象包含以下字段：
   - category: 建议类别（内容/格式/技能/项目）
   - priority: 优先级（高/中/低）
   - issue: 问题描述
   - recommendation: 具体改进建议
```

用户提示词（`resume-analysis-user.st`）：

```
# Input Data
请分析以下简历内容，并参考【技术优化基准】给出深度审计报告。

## 候选人简历
---简历内容开始---
{resumeText}
---简历内容结束---

## 技术优化基准 (参考标准)
在提出优化建议时，请务必对标以下高标准场景及表达逻辑：

### 高并发与缓存优化
| 技术场景 | 参考表达 |
|---------|---------|
| 多级缓存 | Redis + Caffeine 两级缓存架构，解决击穿/穿透/雪崩，支撑 30w+ QPS |
| 原子操作 | Redis Lua 脚本实现分布式令牌桶限流或原子库存扣减 |

### 异步与性能调优
| 技术场景 | 参考表达 |
|---------|---------|
| 异步编排 | `CompletableFuture` 对多源 RPC 调用编排，RT 从秒级到百毫秒级 |
| 线程治理 | 动态线程池参数监控与调整，解决父子任务线程池隔离导致的死锁问题 |

### 微服务架构与数据一致性
| 技术场景 | 参考表达 |
|---------|---------|
| 数据同步 | Canal + RabbitMQ/RocketMQ 实现 MySQL 增量数据实时同步至 Elasticsearch |
| 分布式事务 | 基于消息队列（延时消息）实现订单超时关闭或数据最终一致性 |
| 网关与安全 | Spring Cloud Gateway + Spring Security OAuth2 + JWT + RBAC 动态权限控制 |

### 复杂业务建模与设计模式
| 技术场景 | 参考表达 |
```

```
|---------|---------|
| DDD 领域驱动 | 抽象领域模型，运用工厂、策略、模板方法模式构建业务链路 |
| 规则引擎 | 责任链模式处理前置校验，组合模式+决策树支撑复杂业务逻辑 |
| 状态管理 | Spring 状态机管理复杂业务流转（如订单状态），确保幂等性 |

### 稳定性与大数据处理
| 技术场景 | 参考表达 |
|---------|---------|
| 全链路治理 | Sentinel 限流降级、SkyWalking 链路追踪、MAT 分析 Dump 定位内存泄漏 |
| 分库分表 | ShardingSphere 复合分片算法，解决亿级数据量下的查询性能瓶颈 |
| 批处理 | EasyExcel + MyBatis 批处理 + 任务表异步化，优化百万级数据导入导出 |

## 审计维度
| 维度 | 评估标准 |
|------|---------|
| 技术深度 | 是否体现底层原理（如锁机制、索引优化、并发模型） |
| 业务价值 | 是否描述技术如何解决业务痛点（如超卖、卡顿、延迟） |
| 量化结果 | 是否有明确的性能指标（RT、QPS、吞吐量、交付周期） |

## 输出要求
请严格按照 JSON 格式输出分析结果，直接输出 JSON 对象，不要包含任何 Markdown 代码块标签。
```

提示词最佳实践

可以参考这篇文章：手把手教你写出生产级结构化 Prompt。

RAG 与向量存储

本项目使用 Spring AI 的 `VectorStore`（PostgreSQL + pgvector）实现 RAG（检索增强生成），包括文档分块与向量化、相似度检索、前置过滤与有效命中判定、以及结合 ChatClient 的问答流程。**RAG 的完整设计与实现（向量化服务、检索策略、流式 SSE 问答等）已单独成文，请直接查阅：**

Spring AI + pgvector 实现 RAG 知识库问答

流式响应与 SSE

流式响应指大模型边生成边返回，提升长文本与对话场景下的体验。本项目在知识库 RAG 聊天中采用 **SSE（Server-Sent Events）** 推送流式内容。**SSE 的实现细节、协议格式、转义与前端对接等已单独成文，请直接查阅：**

基于 SSE 实现打字机效果输出

虚拟线程：高并发 AI 调用的性能优化

虚拟线程（Virtual Threads）是 Java 19 引入的预览特性，Java 21 正式发布，旨在大幅提高高并发场景下的线程处理能力。

传统线程 vs 虚拟线程

特性	平台线程（传统）	虚拟线程
创建成本	高（约 1MB 栈空间）	极低（约几 KB）
上下文切换	昂贵（内核态切换）	便宜（用户态调度）
最大数量	几千个	百万级
适用场景	CPU 密集型	I/O 密集型

为什么 AI 应用适合虚拟线程？

AI 应用的特点：

- **大量 I/O 等待**：调用大模型 API 需要等待网络响应
- **并发请求多**：多用户同时使用 AI 功能
- **内存敏感**：传统线程模型在高并发下内存消耗巨大

```
传统线程模型：
┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐
|Thread-1 |  |Thread-2 |  |Thread-3 |  |Thread-N |  → 内存消耗巨大
| 1MB栈   |  | 1MB栈   |  | 1MB栈   |  | 1MB栈   |
└──────────┘  └──────────┘  └──────────┘  └──────────┘

虚拟线程模型：
┌──────────────────────────────────────────┐
|         Carrier Thread (F/J Pool)         |  → 内存消耗极小
|  ┌────┐ ┌────┐ ┌────┐ ┌────┐ ┌────┐     |
|  |VT1 | |VT2 | |VT3 | |... | |VTN |     |
|  └────┘ └────┘ └────┘ └────┘ └────┘     |
└──────────────────────────────────────────┘
```

启用虚拟线程配置

```yaml
spring:
  threads:
    virtual:
      enabled: true  # 启用虚拟线程
```

Spring Boot 4.0 + 已内置对虚拟线程的支持，但在本项目中仍需要通过 `spring.threads.virtual.enabled: true` 显式开启（见上面的配置片段与仓库 `application.yml`）。

性能对比

下面的数字仅用于说明“虚拟线程更适合 I/O 密集型”的直觉，实际效果请以你的链路压测为准（尤其是外部 LLM API 的 QPS/并发限制往往是瓶颈）。

线程模型	线程数	内存消耗	吞吐量
传统线程池	200	~200MB	~100 req/s
虚拟线程	1000	~10MB	~500 req/s

注意：虚拟线程主要优化 I/O 密集型场景，对 CPU 密集型任务帮助不大。

生产环境实践

API Key 安全管理

严禁硬编码 API Key，采用环境变量注入：

```yaml
# ❌ 错误做法
spring:
  ai:
    openai:
      api-key: sk-xxxxxxxxxxxxx  # 绝对不要这样写！

# ✅ 正确做法
spring:
  ai:
    openai:
      api-key: ${AI_BAILIAN_API_KEY}  # 从环境变量读取
```

环境变量配置方式：

```bash
# 方式一：终端临时设置
export AI_BAILIAN_API_KEY=sk-xxxxxx

# 方式二：~/.bashrc 或 ~/.zshrc 永久设置
echo 'export AI_BAILIAN_API_KEY=sk-xxxxxx' >> ~/.bashrc

# 方式三：Docker Compose
services:
  app:
    environment:
      - AI_BAILIAN_API_KEY=${AI_BAILIAN_API_KEY}

# 方式四：系统环境变量文件
# .env 文件（记得加入 .gitignore）
AI_BAILIAN_API_KEY=sk-xxxxxx
```

重试策略配置

针对不稳定的网络环境，配置合理的重试策略：

```yaml
spring:
  ai:
    retry:
      max-attempts: 3           # 最多重试 3 次
      backoff:
        initial-interval: 2000  # 初始间隔 2 秒
        multiplier: 2           # 每次重试间隔翻倍
        max-interval: 10000     # 最大间隔 10 秒
```

本项目策略：

```yaml
spring:
  ai:
    retry:
      max-attempts: 1           # 不重试，立即返回错误
      on-client-errors: false   # 客户端错误不重试
```

选择不重试的原因：

- **快速失败**：AI 调用失败通常是业务问题（如 API Key 错误），重试无意义
- **用户体验**：避免长时间等待后仍失败
- **成本控制**：避免重试消耗额外的 API 配额

限流保护

为防止 API 滥用和成本失控，需要实现限流保护：

```java
@PostMapping("/api/interview/sessions")
@RateLimit(dimensions = {RateLimit.Dimension.GLOBAL, RateLimit.Dimension.IP},
count = 5)
public Result<InterviewSessionDTO> createSession(@RequestBody
CreateInterviewRequest request) {
    InterviewSessionDTO session = sessionService.createSession(request);
    return Result.success(session);
}
```

说明（以本项目为准）：

- 限流能力由自定义注解 `@RateLimit` 提供，底层使用 **Redisson + Lua 脚本**实现滑动窗口的原子限流。
- 支持多维度组合（全局/IP/用户）。例如 `GLOBAL + IP` 能在“全站保护”和“单 IP 防刷”之间取得平衡。
- 触发限流时默认抛出 `RateLimitExceededException`，由全局异常处理转换为统一的错误响应（错误码见 `ErrorCode.RATE_LIMIT_EXCEEDED`）。

成本监控

当前仓库未内置“Token 用量/成本”的完整监控组件（例如把 usage 写入时序库）。如果你准备把 demo 推进到生产，建议至少采集：

- **调用维度**：场景（简历分析/出题/评估/RAG）、模型名、是否流式
- **稳定性维度**：成功率、错误码分布（429/5xx/超时/解析失败）
- **性能维度**：端到端耗时（含 p95/p99）
- **成本维度（可选）**：输入/输出 token 与金额（前提是供应商在响应里提供 usage 或你能从 SDK 拿到）

成本优化建议

优化策略	说明	预估节省
Prompt 压缩	移除冗余描述，使用简洁的提示词	20-30%
结果缓存（可选增强）	相同输入返回缓存结果（本项目未启用 Spring Cache，需要你自行引入）	30-50%
分批处理	合并多个小请求为一个大请求	10-20%
模型降级	非关键场景使用更便宜的模型	40-60%

> 提示：如果你要做“AI 结果缓存”，务必同时考虑 **隐私合规（缓存是否包含用户简历/回答）** 与 **失效策略（模型/提示词升级导致缓存过期）**。

最佳实践

分批评估策略

对于大规模面试评估场景，建议采用分批评估策略：

```
private List<BatchEvaluationResult> evaluateInBatches(
    String sessionId,
    String resumeSummary,
    List<InterviewQuestionDTO> questions
) {
    List<BatchEvaluationResult> results = new ArrayList<>();
    for (int start = 0; start < questions.size(); start += evaluationBatchSize)
{
        int end = Math.min(start + evaluationBatchSize, questions.size());
        List<InterviewQuestionDTO> batchQuestions = questions.subList(start,
end);
        EvaluationReportDTO report = evaluateBatch(sessionId, resumeSummary,
batchQuestions, start, end);
        results.add(new BatchEvaluationResult(start, end, report));
    }
    return results;
}
```

配置建议：

- **默认批次大小**：8 题/批（平衡评估质量和 Token 消耗）
- **问题数 ≤ 8**：使用单批评估
- **问题数 9-16**：使用 2 批评估
- **问题数 > 16**：适当增加批次大小或数量

优雅降级设计

在 AI 调用失败时提供备用方案，确保系统可用性。本项目通过 `StructuredOutputInvoker` 封装重试，仍失败后抛出 `BusinessException`；在评估等场景中可在外层 catch 后降级为聚合结果，详见 [模拟面试功能实现](#)。

错误处理

针对 AI 调用可能出现的各种异常，使用 `StructuredOutputInvoker.invoke(...)` 统一封装后，在业务层 catch 并转换为 `BusinessException`（如 `ErrorCode.RESUME_ANALYSIS_FAILED`），由全局异常处理器返回统一错误响应。

RAG 相关实践

Embedding 批次大小限制、向量删除与 pgvector 元数据过滤等实现见 知识库 RAG 问答。

总结

通过本文的实践，你可以快速在 Spring Boot 项目中集成 Spring AI，实现简历评分、面试问题生成、面试评估等 AI 功能；RAG 与 SSE 的完整实现见 知识库 RAG 问答、SSE 流式输出，模拟面试实现建 模拟面试功能实现。

完整代码可参考项目源码中的以下文件（路径相对于 `app/src/main/java/interview/guide/` 与 `app/src/main/resources/`）：

- `modules/resume/service/ResumeGradingService.java` - 简历评分服务
- `modules/interview/service/InterviewQuestionService.java` - 面试问题生成服务
- `modules/interview/service/AnswerEvaluationService.java` - 面试评估服务（分批评估 + 二次总结）
- `common/ai/StructuredOutputInvoker.java` - 结构化输出重试封装
- `resources/prompts/` - 提示词模板文件